Team Name	Wasted Potential									
Team's Domain	Maze design and playing									
Link to Repo:	https://github.com/CSC207-2022F-UofT/course-project-wasted-potential									
Item	**Comments (Milestone 4)**	**Comments(Milestone 5)**	**Rubric**					**Grade(Milestone 4)**	**Grade (Milestone 5)**	**Out of**
			≤ 4/8 Inadequate	**5/8 Acceptable**	**6/8 Good**	**7/8 Excellent**	**8/8 Exceptional**			
Final Presentation	NA for Milestone 4	Best presentation I saw all week. The secondary reviewer told that this was the most impressive presentation he had seen all day and he had seen over 7 that day. Presentation skills were succint and delivery was good. QA was handled well.	Presentation was significantly incomplete and was clearly not rehearsed by the team.	Presentation was minimal in places and limited evidence that the team rehearsed.	Presentation contained the required elements and the team had clearly prepared, but could have been improved in specific ways still.	Presentation was polished, contained the required elements, and was easy to follow for someone unfamiliar with the project.	Presentation went above and beyond expectations. Significant effort went into the slides and demonstrations. Presentation looked professional and was clearly rehearsed.	NA	20	20
Functionality	Program does not compile on submitted commit; you should put your main application class in order. Examining the code, the structure for some of the planned features is present, but the implementation is still missing for many of them. You need to make sure to prioritize getting the overall flow of the program to work for milestone 5! It is important to get features working together rather than keeping on building features.	Solid improvement in functionality. All use cases are completed. Overall, a very impressive result.	- Program does not run or does not meet much of the team's specification. - Program does not store or load state in any meaningful way.	- Program meets only some of the team's specification. Notable parts of the functionality are left unimplemented.	- Program satisfies most of the team's specification.	- Program satisfies the specification and the scope of the project was appropriate given the team's size.	- Program satisfies the specification and the scope of the project is impressive. - Overall, the specification is well-scoped and ambitious given the team size.	6	16	16
Submitted Pull Requests	Pull requests contain significant code contributions but are mostly missing descriptions. Including description will help your teammates more effectively review your changes, it can either be a summary of what your code does or the description of your code. Reviews contained limited feedback and can be improved in terms of talking about clean architecture, testing, SOLID principles etc.	Pull requests involved significant code contributions. Detailed descriptions are added in many. However, feedback is still lacking and was limited.	Pull requests noticeably absent and explanations are minimal.	Some pull requests demonstrate adequate contributions and have reasonable explanations accompanying them.	All pull requests demonstrate adequate contributions and have reasonably high quality explanations accompanying them.	Almost all pull requests demonstrate significant contributions and have mostly high quality explanations accompanying them.	All pull requests demonstrate significant contributions and have high quality explanations accompanying them.	6	7.5	8
Code Organization	Based on Clean Architecture code is divided into packages. To improve this make sure that naming conventions are consistent across packages(all lower case).	No new changes in the architecure. Feedback taken into account.	- Package structure and code organization needs a complete refactoring.		- Package structure or code organization could use some work to be more organized and to make it easier to find things, but overall well done.		- Code is organized in a meaningful way. Easy to follow package structure. Clear that significant thought was put into how the code was organized.	6	8	8
Testing	I think your group is already aware of your lack of tests. I would recommend shifting to TDD , if you don't want to go fully TDD, make sure each PR has at least some automated or manual tests.	Exponential increase in testing. Almost above 90% coverage in all classes.	- A clear lack of tests. Group needs to increase test coverage significantly.	- There is reasonable test coverage and an effort has been made to incorporate testing into the project, but there is clear room for further improvement.	- Tests cover much of the code, but some tests are missing for important parts of the program. Some evidence of test coverage is documented, but there is room to improve this.	- Tests cover most of the code/components. - Evidence of code coverage and relevant documentation present. - Documentation of code coverage MUST be clearly provided in the repo to earn this mark or higher.	- Tests cover practically all of the code/components. - Evidence of code coverage and documentation of what wasn't tested (and why) is present and of high quality. - Untested parts that could be tested are indicated in issues for future work.	0	8	8
Code Style and Documentation	There are still quite a few Java warnings that should be addressed. About 122 warnings with mixture of weak warnings can be found (to find these go to Code -> Inspect Code). Some Javadoc and comments are there but many methods still have documentation to be added. Please fix some of the naming convention issues (make all lower case) and do the required changes before Milestone 5.	Good effort in reducing the warnings. I am impressed by the significant improvement. Relevant java doc added where needed. Naming conventions fixed.	- Warnings in IntelliJ that need to be addressed. - Javadoc used infrequently or improperly. - Hard to read the code. - Significant issues with naming conventions and styling. - Little to no comments where they should be included for clarity.	- Notable warnings in IntelliJ that have yet to be addressed. - Javadoc present throughout some of the code, with clear room to improve. - Few errors in naming convention and styling present in the code. - Lack of comments where they would be helpful.	- Practically all warnings in IntelliJ have been addressed. - Javadoc present throughout the code where applicable, with only minor issues present. - Few errors in naming convention and styling present in the code. - Some comments missing where they would help the reader better understand the code.	- Practically all warnings in IntelliJ have been addressed. - Javadoc used properly to document relevant code. - Little to no errors in naming convention and styling present in the code. - Comments included in the code where appropriate.	- All warnings in IntelliJ have been addressed. - Javadoc used properly to document relevant code. - No errors in naming convention and styling present in the code. - Code style is consistent across the entire code base and appropriate comments are present throughout the code. - One could easily navigate the Javadoc and understand how to make use of the team's code.	5	7	8
Use of GitHub Features	GitHub feature use is standard, branches and PRs have been used, but can be used more effectively to effectively track and develop. Something that you can implement is the GitHub Action for the Gradle test suite or something else. I would advice you to research what you can use here if possible.	No new additional features used. Action remain same as before.	- Group made little to no effort in using GitHub features such as pull requests, issues and actions. Significant room for improvement in how Issues and Projects are being used.	- Group minimally used GitHub features and there is significant room to improve. - Issues and Projects could be used more effectively to document current and future work.	- Group used some GitHub features (e.g. pull requests, actions, issues, Projects) but could incorporate more into their workflow in future. - Issues and Projects could be used more effectively to document current and future work.	- Group effectively used several GitHub features (e.g. pull requests, actions, issues, Projects) in ways consistent with what was taught in the course. There is still room to make use of other features though.	- Group effectively used several GitHub features (e.g. pull requests, actions, issues) and went beyond what was expected of them to learn new features, which were incorporated into the team's workflow.	6	7	8
Design Patterns & Code Smells	For the design pattern I would encourage you to document any design patterns you have incorporated and/or document any design patterns you would apply if you had more time. I ran SonarLint plugin and found a lot of code smell issues like match regex, use of contants etc. I would advice use you to install this plugin and resolve the issues. It gives you a summary report.	Solid attempt at taking the feedback from Milestone 4 and removing the code smells. Clear explaining and rational behind use of design pattern provided in submission.	- Design patterns not correctly implemented and/or are not appropriate. - Obvious places where a design pattern could have been applied. - Team has some notable code smells that need to be addressed	- Some attempt was made to incorporate design patterns into the project. - Chosen design patterns were possibly not the right choice for the problem being solved and were forced into the program. - Documentation of where the patterns were used was somewhat minimal in pull requests. - Obvious places to apply design patterns may still be present in the code. - Code smells present in the code which should be addressed.	- A good attempt was made to incorporate design patterns into the project. - Documentation of where the patterns were used was reasonably well done in pull requests. - Obvious places to apply design patterns may still be present in the code. - Some minor code smells remain which the team can work to address.	- Use of design patterns is present and appropriate. - Easy to identify where the pattern was used and was documented in a pull request. - No obvious places where a design pattern could be applied. - Little to no code smells are present in the code.	- Exceptional use of design patterns throughout the project. It is evident that the team went above and beyond to seek out suitable design patterns for their project. - Easy to identify where the pattern was used and this was clearly documented in a pull request. - No obvious places where a design pattern could be applied. - Practically no code smells are present in the code and any that do remain have been flagged as Issues and noted for future work.	5	7.5	8
Clean Architecture	Based on Clean Architecture code is divided into respective packages, minor violations present but can be improved. Also, make sure that naming conventions are consistent across packages(all lower case).	Clean implementation of Clean Architechture and good understanding on why you used them. Would recommend adding the file from submission to Github page. Though seems like some entities are getting out into the Controllers, which you shouldn't be doing. E.g., HintGenerateControl.generateHint takes in GameState as a parameter.	- Project has few attempts at following Clean Architecture. Dependency Rule clearly violated in the code.	- Project somewhat adheres to Clean Architecture, but there are significant violations of the Dependency Rule.	- Project largely adheres to Clean Architecture. It is clear which layer most classes belong to and how they interact with each other. No significant violations of the Dependency Rule.	- Project almost completely adheres to Clean Architecture. It is clear which layer most classes belong to and how they interact with each other. No significant violations of the Dependency Rule.	- Project adheres to Clean Architecture. It is clear which layer each class belongs to and how they interact with each other. No violations of the Dependency Rule.	6	7.5	8
SOLID	The project roughly follows SOLID principles, I did not spot any major violation. With the current organization, I believe the program can be easily extended in meaningful ways. Also, you should document an example of some natural way to extend the functionality and explain how your design would allow you to easily do so. This could be a written explanation in an issue or readme — or part of the final presentation to get full marks in the final stage of development.	Clear understanding of SOLID principles and why you used them. Again would recommend adding the file from submission to Github page.	- Project makes little attempt to follow the SOLID principles. Major violations are present.	`- Project satisfies the SOLID principles in places, but there are some clear violations present. The program could be extended to some degree.	- Project largely satisfies some SOLID principles, with some small exceptions. The program could be extended.	- Project almost completely satisfies the SOLID principles throughout. It would be easy to extend the program in meaningful ways.	- Project completely satisfies the SOLID principles throughout. It would be easy to extend the program in meaningful ways.	5	8	8
Total								45	96.5	100
Additional Feedback:	**(Milestone 4) Nice work ! Please fix the above mentioned issues ! I think your group is already aware of these ! These marks can be improved significantly for Milestone 5. Let's discuss in lab about this.**	**(Milestone 5) It was a great experience mentoring your group! The end result was great and I hope you guys continue on this project. Best of luck for your finals !**								